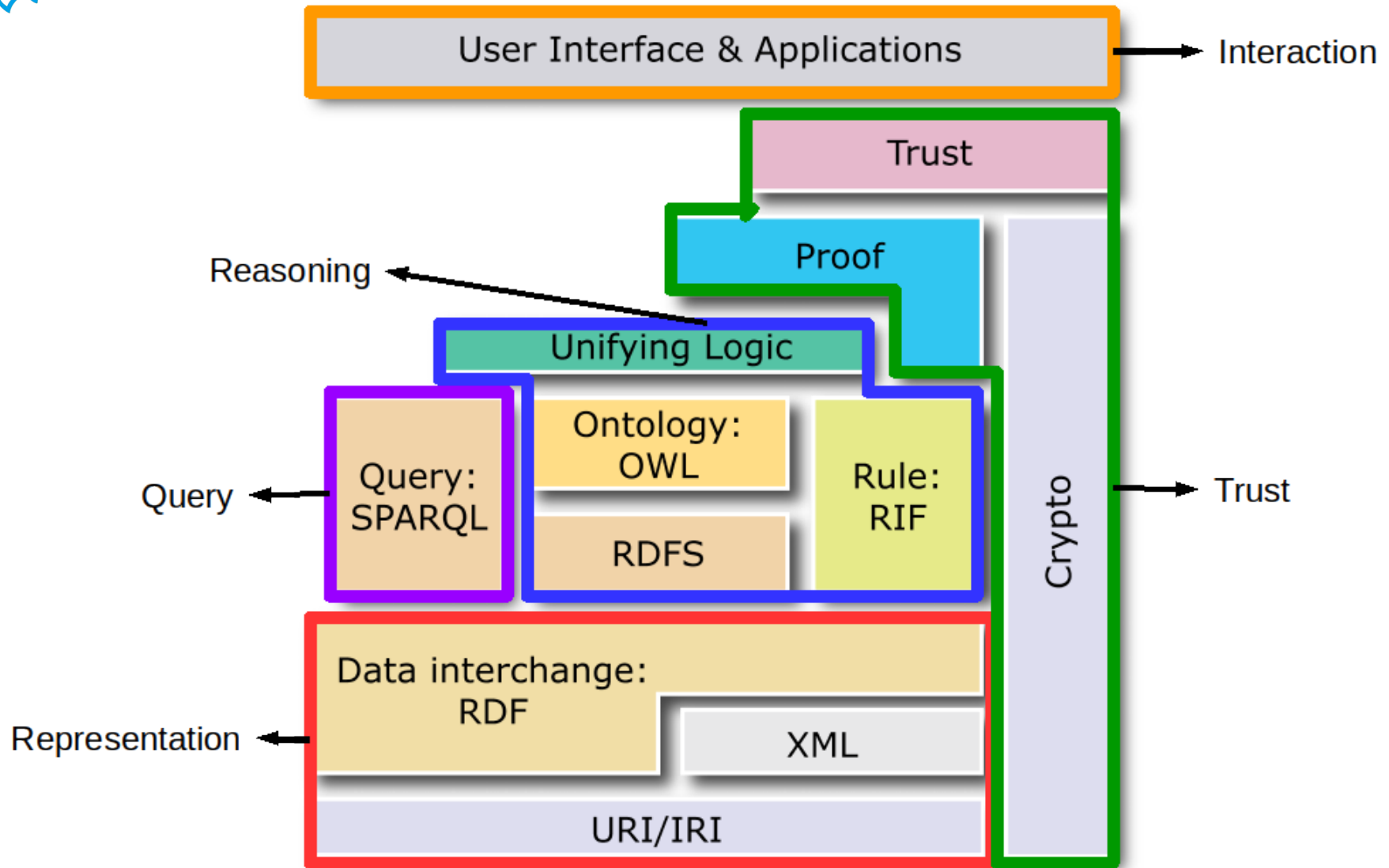




Semantic Web Technology – OWL and Ontology

Knowledge Base and Reasoning

Dewi Wardani





RDFS*

Shortcomings of RDF(S):

- “weak” semantics:

ex:speaksWith rdfs:domain ex:Homo .

ex:Homo rdfs:subClassOf ex:Primates .

does not entail

ex:speaksWith rdfs:domain ex:Primates .

- **expressibility**: no negative information can be specified, no cardinality, disjunction ...



- RDF has a limitation in expressing predicates, which is limited in binary ground predicates. For example, RDF only can provide a predicate “invent” or “wasInvented” and cannot define the relationship between those properties
- RDFS is limited to hierarchy either in a class or a property.



Ontologies and OWL

We need vocabularies which satisfy **some requirements**:

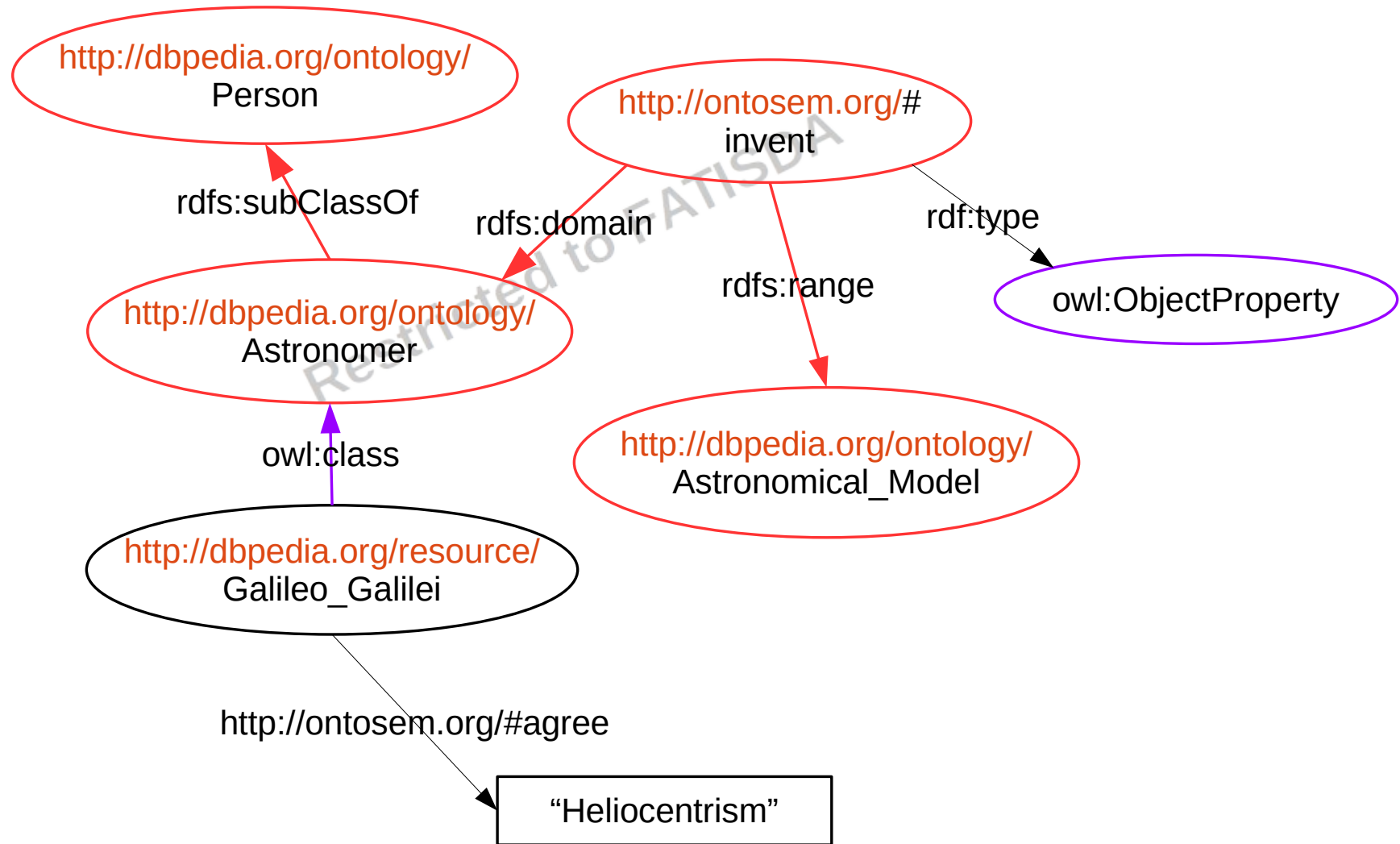
- **A well-defined syntax** that is clear, and known from the area of programming languages.
- **A formal semantics** that describes the meaning of data precisely.
- The semantics **should be consistent**. Hence, it **supports automatic and consistent reasoning**.
- **The sufficient expressive power that offers additional features of expressiveness**, e.g., boolean, disjoint, specialty, cardinality, scope, etc.



- An ontology is an **explicit specification of a conceptualization**.
- Ontologies usually work together with **their languages, e.g., OWL**. Ontologies are vocabularies as same as RDFS.
- They provide more **advanced interoperability** compared to RDFS. RDF and RDFS represent data within objects and relations between objects.
- Meanwhile, **ontologies represent domain data and its interconnections**. Conceptualization is the advanced benefit of ontologies.



- Ontologies use the default syntax based on **RDF/XML**. The meaning (semantics) of syntax and its expression must be clear in an ontology.
- The group of similar semantics (conceptualization) is formed in several classes.
- The taxonomy of classes is formed hierarchically.
- Inference of ontology can create explicit meaning from implicit data.





The components of ontologies:

- **Classes:** Classes or concepts are an abstract group, it is **sets or collections of objects**. **Objects** can be formed as **individuals or classes**. Classes are represented in the hierarchical taxonomy. Ex: Person, Astronomer, Astronomical_Model
- **Individuals:** individuals or instances are **the ground level components of an ontology**. The example of class Person, there is an instance “Galileo_Galilei”. We can write it, **Person (Galileo_Galilei)**. The other example is **Astronomical_Model (Heliocentrism)**.



- **Attributes:** attributes describe objects in an ontology. For example, attributes of an instance Galileo_Galilei, whose attribute "hasGender". hasGender describes the instance Galileo_Galilei.
- **Relationships:** relationship or association expresses how objects in an ontology are connected or related to each other. For example, "subClassOf" is the relationship between Astronomer and Person. The other example is "invent", as a relationship between Astronomer and Astronomical_Model.
- **Restrictions:** restrictions or rules are a part of a relationship and can be attached to it. They are used to customizing of each relationship. For example, 1 Astronomer can invent 1 or N Astronomical_Model, 1 Astronomical_Model has minimal 1 Astronomer, etc



Types of ontologies (coverage):

- **General Ontology/Upper ontology**: this type of ontology describes very general concepts that are the same across all data domains. Ex: schema.org, dbpedia.
- **Domain Ontology**: this type of ontology describes concepts within a specific domain. Ex: GoodRelation (e-commerce), FOAF (social)



The Web Ontology Language (OWL)

- An ontology language is used as a specific language to formalize and to specify ontologies. One of the ontology languages is OWL.
- The other examples of ontology languages are Ontology Interface Layer (OIL), DARPA Agent Markup Language (DAML) and Knowledge Interchange Format (KIF).
- Currently, only OWL is a part of the W3C recommendation



- The last version of OWL is OWL 2.
- One of the important features of OWL is **owl:ObjectProperty**, which defines the types of properties as a class.
- Therefore, it makes the schema of OWL hierarchical.
- In short, **OWL provides formal representation for ontologies.**
- The concept of OWL is based on First Order Logic (**FOL**) and Description Logic (**DL**)



- **OWL Full**
 - contains OWL DL and OWL Lite,
 - is the only OWL sublanguage containing all of RDFS,
 - very expressive,
 - semantically difficult to understand and to work with,
 - undecidable,
 - supported by hardly any software tools.



OWL Full

- **All** OWL language constructs can be used and mixed freely with all RDF(S) language constructs, as long as the resulting document is valid RDF(S).
- Due to the **unrestricted** use of OWL and RDF(S) constructs in OWL Full, several problems arise, which suggest the design of more restrictive sublanguages
- The use of classes (or roles) as individuals is called **metamodeling**, and it allows us to talk about classes of classes.



- **OWL DL**
 - contains OWL Lite and is contained in OWL Full,
 - decidable,
 - fully supported by most software tools,
 - worst-case computational complexity: NExpTime.



OWL DL

- **Restricted use of RDF(S)** language constructs: Only those RDF(S) language constructs may be used which are specifically allowed for OWL DL. These are essentially all those RDF(S) language constructs which we have used in our examples throughout this chapter. The use of `rdfs:Class` and `rdf:Property` is forbidden.
- **Type separation and declaration:** Type separation must be respected in OWL DL, i.e. it must be clearly distinguished between individuals, classes, abstract roles, concrete roles, datatypes, and the ontology characteristics specified in the header. In addition, classes and roles must be declared explicitly.
- **Restricted use of concrete roles:** The role characteristics `owl:inverseOf`, `owl:TransitiveProperty`, `owl:InverseFunctionalProperty`, as well as `owl:SymmetricProperty` must not be used for concrete roles
- **Restricted use of abstract roles:** Cardinality restrictions expressed via `owl:cardinality`, `owl:minCardinality`, or via `owl:maxCardinality` must not be used with transitive roles, inverses of transitive roles, or superroles of transitive roles.



- **OWL Lite**
 - contained in OWL DL and OWL Full,
 - decidable,
 - less expressive,
 - worst-case computational complexity: ExpTime.



OWL Lite

- **All restrictions imposed for OWL DL** must be respected.
- **Restricted use of class constructors:** owl:unionOf, owl:complementOf, owl:hasValue, owl:oneOf, owl:disjointWith, owl:DataRange must not be used.
- **Restricted use of cardinality restrictions:** They can only be used with the numbers 0 and 1.
- **Mandatory use of class names:** In some situations, class names must be used: in the subject of owl:equivalentClass and rdfs:subClassOf, in the object of rdfs:domain.
- **Mandatory class names or role restrictions:** In some situations, class names or role restrictions must be used: in the object of owl:equivalentClass, rdfs:subClassOf, rdf:type, owl:allValuesFrom, owl:someValuesFrom, rdfs:range. Additionally, owl:intersectionOf must be used only for class names and role restrictions.



contains information about namespaces,
versioning, and so-called annotations

Header

```
<?xml version="1.0"?>
<!DOCTYPE Ontology [
  <!ENTITY xsd "http://www.w3.org/2001/XMLSchema#" >
  <!ENTITY xml "http://www.w3.org/XML/1998/namespace" >
  <!ENTITY rdfs "http://www.w3.org/2000/01/rdf-schema#" >
  <!ENTITY rdf "http://www.w3.org/1999/02/22-rdf-syntax-ns#" >
]>
<Ontology xmlns="http://www.w3.org/2002/07/owl#"
  xml:base="http://www.example.org/ex/"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:xml="http://www.w3.org/XML/1998/namespace"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  ontologyIRI="http://www.example.org/">
  <Prefix name="ex" IRI="http://www.example.org/#"/>
  <Prefix name="owl" IRI="http://www.w3.org/2002/07/owl#"/>
  <Prefix name="rdf" IRI="http://www.w3.org/1999/02/22-rdf-syntax-ns#"/>
  <Prefix name="xsd" IRI="http://www.w3.org/2001/XMLSchema#"/>
  <Prefix name="rdfs" IRI="http://www.w3.org/2000/01/rdf-schema#"/>
  <Prefix name="ex" IRI="http://www.example.org/#"/>
```



```
<Declaration>
  <Class IRI="#Astronomer"/>
</Declaration>
<Declaration>
  <Class IRI="#Astronomical_Model"/>
</Declaration>
<Declaration>
  <Class IRI="#Person"/>
</Declaration>
<Declaration>
  <ObjectProperty IRI="#invent"/>
</Declaration>
<Declaration>
  <DataProperty IRI="#agree"/>
</Declaration>
<Declaration>
  <NamedIndividual IRI="#Galileo_Galilei"/>
</Declaration>
```

Declarations

```
<SubClassOf>
  <Class IRI="#Astronomer"/>
  <Class IRI="#Person"/>
</SubClassOf>
<ClassAssertion>
  <Class IRI="#Astronomer"/>
  <NamedIndividual IRI="#Galileo_Galilei"/>
</ClassAssertion>
<DataPropertyAssertion>
  <DataProperty IRI="#agree"/>
  <NamedIndividual IRI="#Galileo_Galilei"/>
  <Literal datatypeIRI="#&rdfs;Literal">Heliocentrism</Literal>
</DataPropertyAssertion>
<ObjectPropertyDomain>
  <ObjectProperty IRI="#invent"/>
  <Class IRI="#Astronomer"/>
</ObjectPropertyDomain>
<ObjectPropertyRange>
```



- Each class, property, or individual needs to be declared.
- Same URI can stand e.g. for both an individual and a class:
:John rdf:type :Father .
:Father rdf:type :SocialRole .
- Semantics: This is semantically interpreted as if the two occurrences of Father were in fact distinct.



- The script shows that **OWL also includes rdf and rdfs vocabularies.**
- OWL categorizes properties into two types: **owl:ObjectProperty and owl:DatatypeProperty**. The type of the property “http://ontosem.org/#invent” is owl:ObjectProperty and the type of the property “http://ontosem.org/#agree” is owl:DatatypeProperty.
- Some other expressive regulations for properties are owl:sameAs, owl:inverseOf and owl:hasValue etc.
- **The complete version is listed as the W3C Recommendation**



Classes, Roles, and Individuals

- The basic building blocks of OWL are classes and properties/roles, and individuals.

- **Simple class relation**

```
<owl:Class rdf:about="Professor">
```

```
<rdfs:subClassOf rdf:resource="FacultyMember" />
```

```
</owl:Class>
```

- There are two different kinds of roles in OWL: abstract roles and concrete roles. Abstract roles connect individuals with individuals. Concrete roles connect individuals with data values, i.e. with elements of datatypes.



- **Relation among individuals**

```
<owl:Class rdf:about="Professor">  
<rdfs:subClassOf rdf:resource="FacultyMember" />  
</owl:Class>  
<owl:Class rdf:about="Book">  
<rdfs:subClassOf rdf:resource="Publication" />  
</owl:Class>  
<owl:Class rdf:about="FacultyMember">  
<owl:disjointWith rdf:resource="Publication" />  
</owl:Class>
```

Professor and Book are also disjoint.



```
<owl:Class rdf:about="Man">  
<rdfs:subClassOf rdf:resource="Person" />  
</owl:Class>  
<owl:Class rdf:about="Person">  
<owl:equivalentClass rdf:resource="Human" />  
</owl:Class>
```

Man is a subclass of Human.



<Book rdf:about="http://semantic-web-book.org/uri">

<author rdf:resource="markusKroetzsch" />

<author rdf:resource="sebastianRudolph" />

</Book>

<owl:Class rdf:about="Book">

<rdfs:subClassOf rdf:resource="Publication" />

</owl:Class>

http://semantic-web-book.org/uri is a Publication.



```
<Professor rdf:about="rudiStuder" />  
<rdf:Description rdf:about="rudiStuder">  
<owl:sameAs rdf:resource="professorStuder" />  
</rdf:Description>
```

professorStuder is in the class Professor.



<owl:AllDifferent>

<owl:distinctMembers rdf:parseType="Collection">

<Person rdf:about="rudiStuder" />

<Person rdf:about="dennyVrandecic" />

<Person rdf:about="peterHaase" />

</owl:distinctMembers>

</owl:AllDifferent>

3 names are different



- OWL provides language elements for logical and, or, and not, i.e. conjunction, disjunction, and negation.
- They are expressed via owl:intersectionOf, owl:unionOf, and owl:complementOf, respectively. These constructors allow us to combine atomic classes – i.e. class names – to complex classes.



```
<owl:Class rdf:about="SecretariesOfStuder">  
<owl:oneOf rdf:parseType="Collection">  
<Person rdf:about="giselaSchillinger" />  
<Person rdf:about="anneEberhardt" />  
</owl:oneOf>  
</owl:Class>
```

giselaSchillinger and anneEberhardt are the only individuals in the class SecretariesOfStuder



```
<Person rdf:about="anupriyaAnkolekar" />  
  <owl:AllDifferent>  
    <owl:distinctMembers rdf:parseType="Collection">  
      <Person rdf:about="anneEberhardt" />  
      <Person rdf:about="giselaSchillinger" />  
      <Person rdf:about="anupriyaAnkolekar" />  
    </owl:distinctMembers>  
  </owl:AllDifferent>
```

- **anupriyaAnkolekar is not in the class SecretariesOfStuder.**



```
<owl:Class rdf:about="SecretariesOfStuder">  
  <owl:intersectionOf rdf:parseType="Collection">  
    <owl:Class rdf:about="Secretaries" />  
    <owl:Class rdf:about="MembersOfStudensGroup" />  
  </owl:intersectionOf>  
</owl:Class>
```

- all instances of the class SecretariesOfStuder are also in the class Secretaries



```
<owl:Class rdf:about="Professor">  
  <rdfs:subClassOf>  
    <owl:Class>  
      <owl:unionOf rdf:parseType="Collection">  
        <owl:Class rdf:about="ActivelyTeaching" />  
        <owl:Class rdf:about="Retired" />  
      </owl:unionOf>  
    </owl:Class>  
  </rdfs:subClassOf>  
</owl:Class>
```

- every Professor is in at least one of the classes ActivelyTeaching and Retired.



- owl:complementOf, which corresponds to logical negation: The complement of a class consists of exactly those objects which are not members of the class itself

```
<owl:Class rdf:about="Male">
```

```
<owl:complementOf rdf:resource="Female" />
```

```
</owl:Class>
```

```
<Penguin rdf:about="tweety" />
```

- tweety is an instance of Female. However, it can also not be concluded that tweety is not Female, and hence it cannot be concluded that tweety is Male.



Role Restrictions

```
<owl:Class rdf:about="Exam">  
<rdfs:subClassOf>  
<owl:intersectionOf rdf:parseType="Collection">  
<owl:Restriction>  
<owl:onProperty rdf:resource="hasTopic" />  
<owl:minCardinality rdf:datatype="&xsd;nonNegativeInteger"> 3 </owl:minCardinality>  
</owl:Restriction>  
<owl:Restriction>  
<owl:onProperty rdf:resource="hasTopic" />  
<owl:maxCardinality rdf:datatype="&xsd;nonNegativeInteger"> 3 </owl:maxCardinality>  
</owl:Restriction>  
</owl:intersectionOf>  
</rdfs:subClassOf>  
</owl:Class>
```



```
<owl:Class rdf:about="ExamStuder">
```

```
  <owl:equivalentClass>
```

```
    <owl:Restriction>
```

```
      <owl:onProperty rdf:resource="hasExaminer" />
```

```
      <owl:someValuesFrom>
```

```
        <owl:oneOf rdf:parseType="Collection">
```

```
          <owl:Thing rdf:about="rudiStuder" />
```

```
        </owl:oneOf>
```

```
      </owl:someValuesFrom>
```

```
    </owl:Restriction>
```

```
  </owl:equivalentClass>
```

```
</owl:Class>
```



Role Relationships

```
<Exam rdf:about="semanticWebExam">  
<hasExaminer rdf:resource="rudiStuder" />  
</Exam>  
  
<owl:ObjectProperty rdf:about="hasExaminer">  
<rdfs:subPropertyOf rdf:resource="hasParticipant" />  
</owl:ObjectProperty>  
  
<owl:ObjectProperty rdf:about="hasParticipant">  
<owl:equivalentProperty rdf:resource="hasAttendee" />  
</owl:ObjectProperty>  
  
<owl:ObjectProperty rdf:about="hasAttendee">  
<owl:inverseOf rdf:resource="participatesIn" />  
</owl:ObjectProperty>
```



Role Characteristics

```
<owl:Class rdf:about="&owl;Thing">  
<rdfs:subClassOf>  
<owl:Restriction>  
<owl:onProperty rdf:resource="isMemberOf" />  
<owl:allValuesFrom rdf:resource="Organization" />  
</owl:Restriction>  
</rdfs:subClassOf>  
</owl:Class>  
<number rdf:about="five">  
<isMemberOf rdf:resource="PrimeNumbers" />  
</number>
```

- infer that PrimeNumbers is an Organization



- **Symmetry states:** if A and B are in a symmetric role relationship, then B and A (in reverse order) are also in the same role relationship.
- **Transitivity means:** if A and B are in some transitive role relationship, and B and C are in the same role relationship, then A and C are also related via the same role.
- **Functionality of a role means:** if A and B are related via a functional role, and A and C are related by the same role, then B and C are identical in the sense of owl:sameAs
- **Inverse functionality of a role R** is equivalent to the inverse of R being functional.



OWL - Individual

- Class : Woman
- Individual: Mary type Woman
- Class: Man
- Class : Father
- Individual: Bill, Jack, James, Jim, John type man
- James is the same as Jim
- James hasWife Mary
- Jack hasAge 30 (nonnegatifInte..)



OWL – Class, Properties

```
:Woman rdfs:subClassOf :Person .
```

```
:Person owl:equivalentClass :Human .
```

```
[  
  rdf:type      owl:AllDisjointClasses ;  
  owl:members ( :Woman :Man ) .  
]
```

$\text{Woman} \sqcap \text{Man} \sqsubseteq \perp$

```
:hasWife rdfs:subPropertyOf :hasSpouse .
```

```
:hasWife rdfs:domain :Man ;  
        rdfs:range  :Woman .
```



OWL – Complex Classes

```
:Mother owl:equivalentClass [  
  rdf:type owl:Class ;  
  owl:intersectionOf ( :Woman :Parent )  
] .
```

Mother $\equiv$ Woman $\sqcap$ Parent

```
:Parent owl:equivalentClass [  
  rdf:type owl:Class ;  
  owl:unionOf ( :Mother :Father )  
] .
```

Parent $\equiv$ Mother $\sqcup$ Father

```
:ChildlessPerson owl:equivalentClass [  
  rdf:type owl:Class ;  
  owl:intersectionOf ( :Person [ owl:complementOf :Parent ] )  
] .
```

ChildlessPerson $\equiv$ Person $\sqcap \neg$ Parent

```
:Grandfather rdfs:subClassOf [  
  rdf:type owl:Class ;  
  owl:intersectionOf ( :Man :Parent )  
] .
```



OWL - Restrictions

```
:Parent owl:equivalentClass [  
  rdf:type          owl:Restriction ;  
  owl:onProperty   :hasChild ;  
  owl:someValuesFrom :Person  
] .
```

Parent $\equiv \exists \text{hasChild}.\text{Person}$

```
:Orphan owl:equivalentClass [  
  rdf:type          owl:Restriction ;  
  owl:onProperty   [ owl:inverseOf :hasChild ] ;  
  owl:allValuesFrom :Dead  
] .
```

Orphan $\equiv \forall \text{hasChild}^{\neg}.\text{Dead}$



OWL - Restrictions

```
:JohnsChildren owl:equivalentClass [  
  rdf:type owl:Restriction ;  
  owl:onProperty :hasParent ;  
  owl:hasValue :John  
] .
```

JohnsChildren $\equiv \exists \text{hasParent}.\{\text{John}\}$

```
:NarcisticPerson owl:equivalentClass [  
  rdf:type owl:Restriction ;  
  owl:onProperty :loves ;  
  owl:hasSelf "true"^^xsd:boolean .  
] .
```

NarcisticPerson $\equiv \exists \text{loves}.\text{Self}$



OWL - Restrictions

≤ 4 hasChild.Parent (John)

```
:John rdf:type [  
  rdf:type                owl:Restriction ;  
  owl:maxQualifiedCardinality "4"^^xsd:nonNegativeInteger ;  
  owl:onProperty          :hasChild ;  
  owl:onClass              :Parent  
] .
```

≥ 2 hasChild.Parent (John)

```
:John rdf:type [  
  rdf:type                owl:Restriction ;  
  owl:minQualifiedCardinality "2"^^xsd:nonNegativeInteger ;  
  owl:onProperty          :hasChild ;  
  owl:onClass              :Parent  
] .
```

$= 3$ hasChild.Parent (John)

```
:John rdf:type [  
  rdf:type                owl:Restriction ;  
  owl:qualifiedCardinality "3"^^xsd:nonNegativeInteger ;  
  owl:onProperty          :hasChild ;  
  owl:onClass              :Parent  
] .
```



OWL - Restrictions

```
:John  rdf:type  [  
  rdf:type      owl:Restriction ;  
  owl:cardinality  "5"^^xsd:nonNegativeInteger ;  
  owl:onProperty   :hasChild  
]
```

=5 hasChild.T (John)

```
:MyBirthdayGuests  owl:equivalentClass  [  
  rdf:type      owl:Class ;  
  owl:oneOf    ( :Bill  :John  :Mary )  
]
```

MyBirthdayGuests $\equiv$ {Bill, John, Mary}



OWL - Properties

```
:hasParent owl:inverseOf :hasChild .
```

```
:Orphan owl:equivalentClass [  
  rdf:type owl:Restriction ;  
  owl:onProperty [ owl:complementOf :hasChild ] ;  
  owl:allValuesFrom :Dead  
] .
```

Orphan $\equiv \forall \text{hasChild}^c . \text{Dead}$

```
:hasSpouse rdf:type owl:SymmetricProperty .
```

```
:hasChild rdf:type owl:AsymmetricProperty .
```

```
:hasParent owl:propertyDisjointWith :hasSpouse .
```

```
:hasRelative rdf:type owl:ReflexiveProperty .
```

```
:parentOf rdf:type owl:IrreflexiveProperty .
```

```
:hasHusband rdf:type owl:FunctionalProperty .
```

```
:hasHusband rdf:type owl:InverseFunctionalProperty .
```

```
:hasAncestor rdf:type owl:TransitiveProperty .
```



OWL - Datatype

```
:personAge owl:equivalentClass
[ rdf:type rdfs:Datatype;
  owl:onDatatype xsd:integer;
  owl:withRestrictions (
    [ xsd:minInclusive "0"^^xsd:integer ]
    [ xsd:maxInclusive "150"^^xsd:integer ]
  )
] .

:minorAge owl:equivalentClass
[ rdf:type rdfs:Datatype;
  owl:intersectionOf (
    :personAge
    [ rdf:type rdfs:Datatype;
      owl:datatypeComplementOf :minorAge ]
  )
] .

:toddlerAge owl:equivalentClass
[ rdf:type rdfs:Datatype;
  owl:oneOf ( "1"^^xsd:integer "2"^^xsd:integer )
] .
```



- New est OWL is OWL 2
- Not much different comparing to OWL 1
(will be as the assignment)



Further Reading

- A Semantic Web Primer. The MIT Press Cambridge, Massachusetts, London, England. Grigoris Antoniou and Frank van Harmelen.
- <http://www.w3.org/TR/owl2-primer/>